

使用 Firebase App Check 和 reCAPTCHA 驗證 Places API 要求

來源：<https://codelabs.developers.google.com/devsite/codelabs/maps-platform/places-firebase-appcheck?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將瞭解如何在向 Places API 提出要求前，使用 Firebase App Check 和 reCAPTCHA 驗證網頁應用程式。

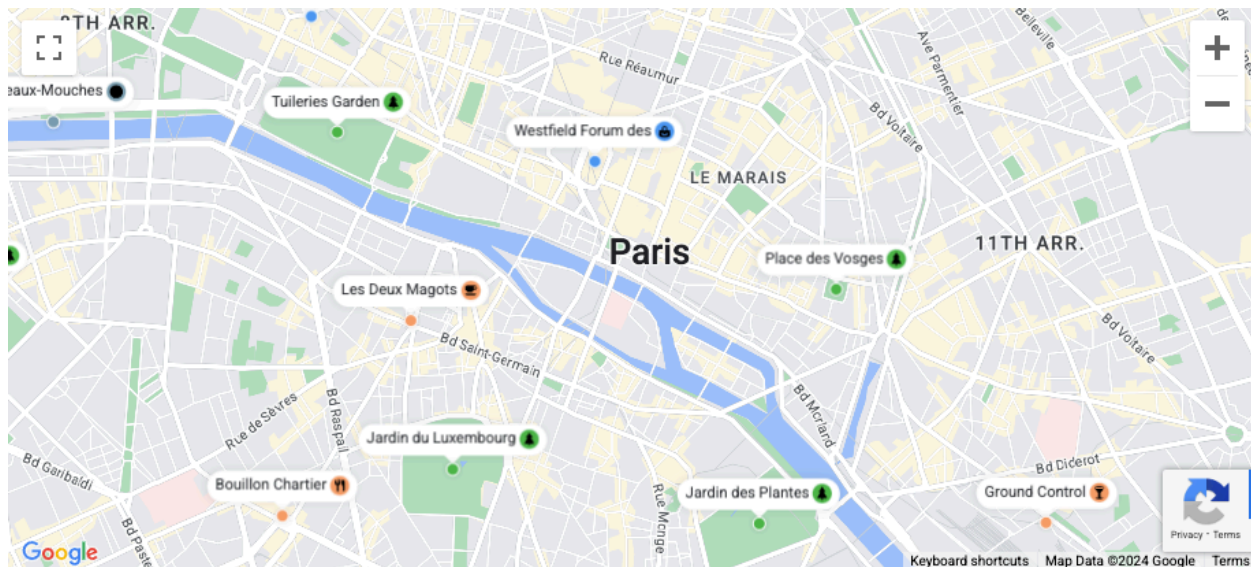
目錄

1. [1. 事前準備](#)
2. [2. 必要條件](#)
3. [3. 進行設定](#)
4. [4. 整合 Firebase App Check](#)
5. [5. 試用版應用程式](#)
6. [6. 程式碼總覽](#)
7. [7. 執行應用程式](#)
8. [8. 這樣就完成了，做得好！](#)

步驟 1 · 約 3 分鐘

1. 事前準備

為確保與網頁應用程式互動的使用者身分合法，您將導入 Firebase App Check，並運用 reCAPTCHA JWT 權杖驗證使用者工作階段。完成這項設定後，您就能安全地處理從用戶端應用程式傳送至 [Places API \(新版\)](#) 的要求。



[即時連結](#)

建構目標

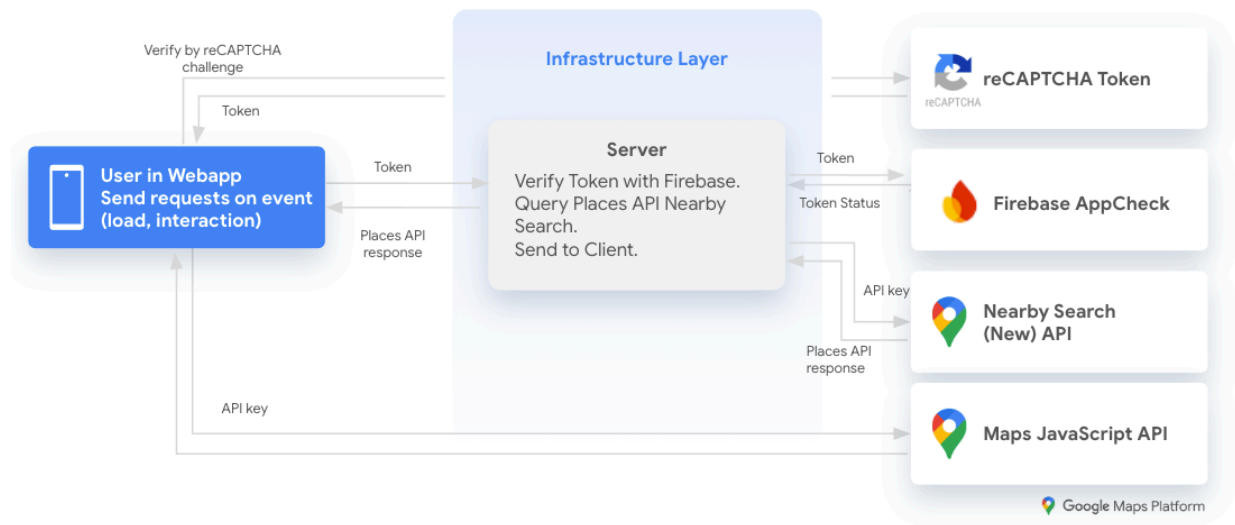
為說明這項功能，您將建立一個網頁應用程式，在載入時顯示地圖。此外，系統也會使用 Firebase SDK 產生 reCAPTCHA 權杖，接著，這個權杖會傳送至 Node.js 伺服器，Firebase 會先驗證權杖，再完成對 Places API 的任何要求。

如果權杖有效，Firebase App Check 會儲存權杖直到過期，因此不必為每個用戶端要求建立新權杖。如果權杖無效，系統會提示使用者再次完成 reCAPTCHA 驗證，以取得新權杖。

步驟 2 · 約 3 分鐘

2. 必要條件

如要完成本程式碼研究室，請先熟悉下列項目。



必要 Google Cloud 產品

- [Google Cloud Firebase App Check](#)：用於管理權杖的資料庫
- [Google reCAPTCHA](#)：建立及驗證權杖。這項工具用於區別網站上的真人和機器人。這項服務會分析使用者行為、瀏覽器屬性和網路資訊，產生分數來指出使用者是機器人的可能性。如果分數夠高，系統會將使用者視為真人，不必採取進一步行動。如果分數偏低，系統可能會顯示 CAPTCHA 謎題，確認使用者身分。與傳統人機驗證方法相比，這種做法對使用者體驗的干擾較小，因此更為順暢。
- (選用) [Google Cloud App Engine](#)：部署環境。

必要 Google Maps Platform 產品

在本程式碼研究室中，您將使用下列 Google 地圖平台產品：

- 已載入並顯示在網頁應用程式中的 [Maps JavaScript API](#)
- 後端伺服器發出的 [Places API \(新版\)](#) 要求

本程式碼研究室的其他規定

如要完成本程式碼研究室，您需要下列帳戶、服務和工具：

- 已啟用計費功能的 Google Cloud Platform 帳戶
- 已啟用 Maps JavaScript API 和 Places 的 Google Maps Platform API 金鑰
- 具備 JavaScript、HTML 和 CSS 的基本知識
- Node.js 的基本知識
- 您選擇的文字編輯器或 IDE

步驟 3 · 約 5 分鐘

3. 進行設定

設定 Google Maps Platform

如果您尚未建立 Google Cloud Platform 帳戶，以及啟用計費功能的專案，請參閱「[開始使用 Google Maps Platform](#)」指南，建立帳單帳戶和專案。

1. 在 [Cloud 控制台](#) 中，按一下專案下拉式選單，然後選取要用於本程式碼研究室的專案。

A blue header bar with a white hamburger menu icon on the left, the text "Google Cloud Platform" in the center, and a white icon of a computer screen with the text "google.com" and a downward arrow on the right.

2. 在 [Google Cloud Marketplace](#) 中，啟用本程式碼研究室所需的 Google Maps Platform API 和 SDK。如要瞭解如何操作，請觀看[這部影片](#)或參閱[這份說明文件](#)。
3. 在 Cloud Console 的「憑證」頁面中產生 API 金鑰。請按照[這部影片](#)或[這份文件](#)中的步驟操作。所有 Google Maps Platform 要求都需要 API 金鑰。

建議您限制 API 金鑰只能用於特定網域。在本程式碼研究室中，本機開發的網址為 `http://localhost:3000/*`，App Engine 部署作業的網址為 `https://yourdomain.eu.r.appspot.com/*`

應用程式預設憑證

您將使用 Firebase Admin SDK 與 Firebase 專案互動，並向 Places API 發出要求，且必須提供有效憑證，才能正常運作。

我們會使用**應用程式預設憑證 (ADC) 驗證**，驗證伺服器是否可發出要求。您也可以建立服務帳戶，並在程式碼中儲存憑證 (不建議)。

定義：應用程式預設憑證 (ADC) 是 Google Cloud 提供的一種機制，可自動驗證應用程式，不必明確管理憑證。這項工具會在各種位置 (例如環境變數、服務帳戶檔案或 Google Cloud 中繼資料伺服器) 尋找憑證，並使用找到的第一個憑證。

- 在終端機中，使用下列指令，讓應用程式代表目前登入的使用者安全地存取 Google Cloud 資源：

```
gcloud auth application-default login
```

- 您會在根目錄建立 `.env` 檔案，指定 Google Cloud 專案變數：

```
GOOGLE_CLOUD_PROJECT="your-project-id"
```

在所有設定中使用相同的 Google Cloud 專案 ID

替代憑證 (不建議)

建立服務帳戶

- 「Google 地圖平台」分頁標籤 > 「+ 建立憑證」 > 「服務帳戶」

- 新增 Firebase AppCheck 管理員角色，然後輸入您剛才輸入的服務帳戶名稱，即：firebase-appcheck-codelab@yourproject.iam.gserviceaccount.com

憑證

- 按一下建立的服務帳戶
 - 前往「金鑰」分頁，依序點選「建立金鑰」>「JSON」，然後儲存下載的 JSON 憑證。將自動下載的 xxx.json 檔案移至根資料夾
 - (下一章) 將其正確命名為 nodejs 檔案 server.js (firebase-credentials.json)
-

步驟 4 · 約 10 分鐘

4. 整合 Firebase App Check

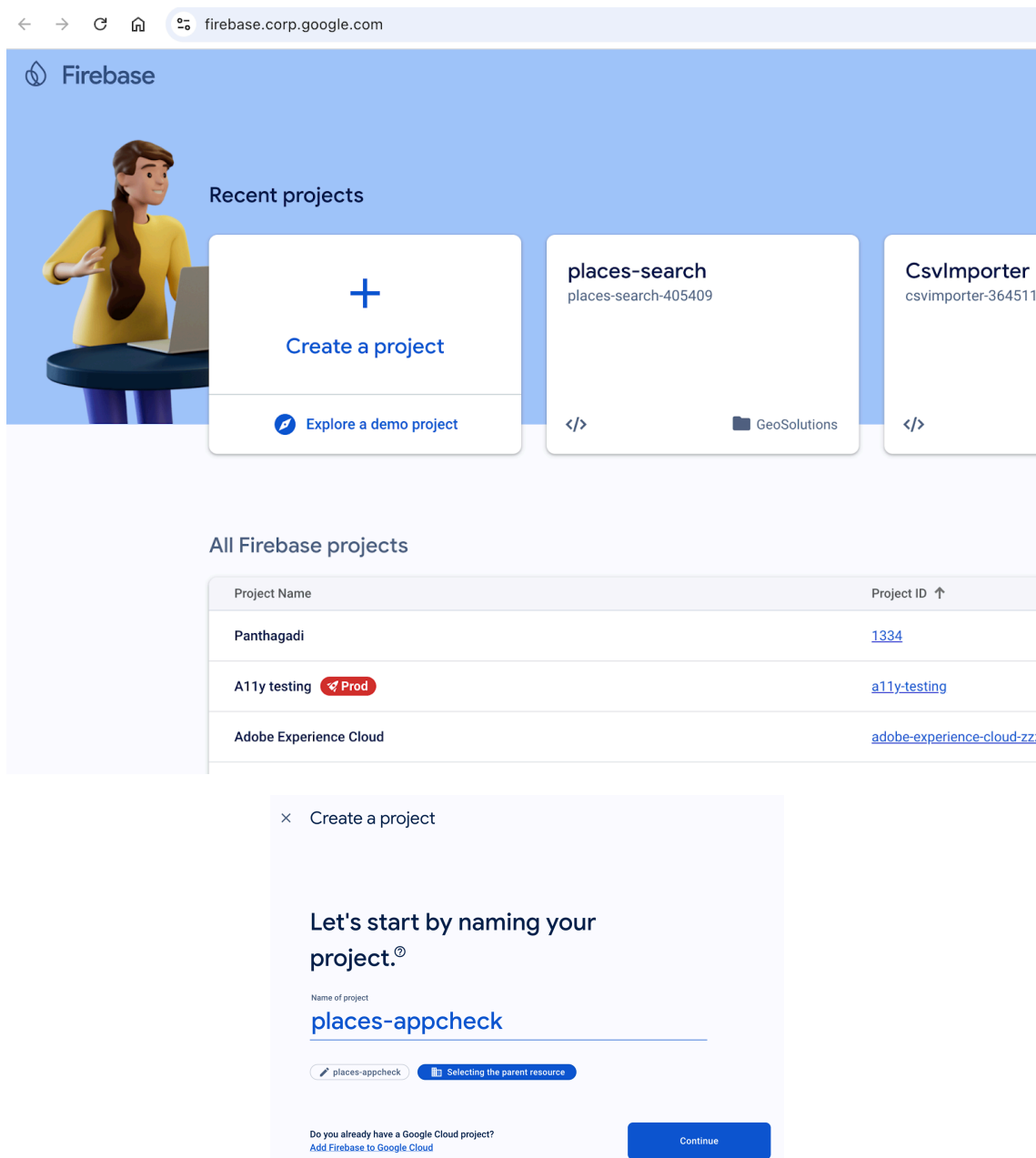
您會取得 Firebase 設定詳細資料和 reCAPTCHA 密鑰。

您會將這些值貼到示範應用程式中，然後啟動伺服器。

在 Firebase 中建立應用程式

- 前往專案管理員 <https://console.firebase.google.com> (尋找連結)：

選取已建立的 Google Cloud 雲端專案 (您可能必須指定「選取父項資源」)



The screenshot shows the Firebase console interface. At the top, there's a navigation bar with the Firebase logo. Below it, the 'Recent projects' section displays a 'Create a project' button with a plus icon and an 'Explore a demo project' button. To the right, there are two project cards: 'places-search' (places-search-405409) and 'CsvImporter' (csvimporter-364511). Below this, the 'All Firebase projects' section shows a table with columns 'Project Name' and 'Project ID'. The table lists four projects: 'Panthagadi' (ID: 1334), 'A11y testing' (ID: a11y-testing, marked as Prod), and 'Adobe Experience Cloud' (ID: adobe-experience-cloud-zzz). Below the table, a modal window titled 'Create a project' is open. It prompts the user to 'Let's start by naming your project.' and shows the 'Name of project' field with the value 'places-appcheck'. There are two tabs: 'places-appcheck' (selected) and 'Selecting the parent resource'. At the bottom of the modal, there's a question 'Do you already have a Google Cloud project?' with a link 'Add Firebase to Google Cloud' and a 'Continue' button.

Project Name	Project ID ↑
Panthagadi	1334
A11y testing Prod	a11y-testing
Adobe Experience Cloud	adobe-experience-cloud-zzz

× Create a project

Let's start by naming your project.®

Name of project

places-appcheck

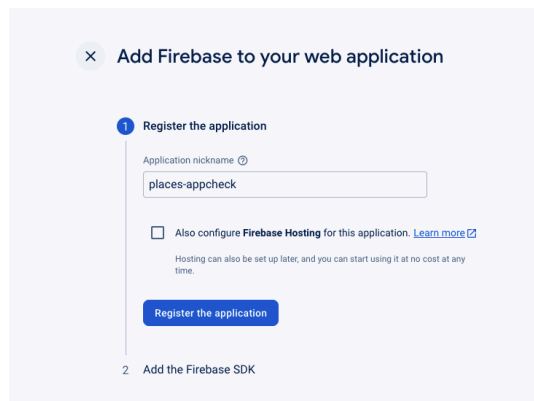
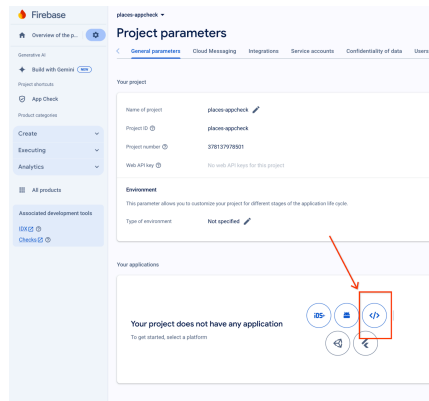
places-appcheck Selecting the parent resource

Do you already have a Google Cloud project?
[Add Firebase to Google Cloud](#)

Continue

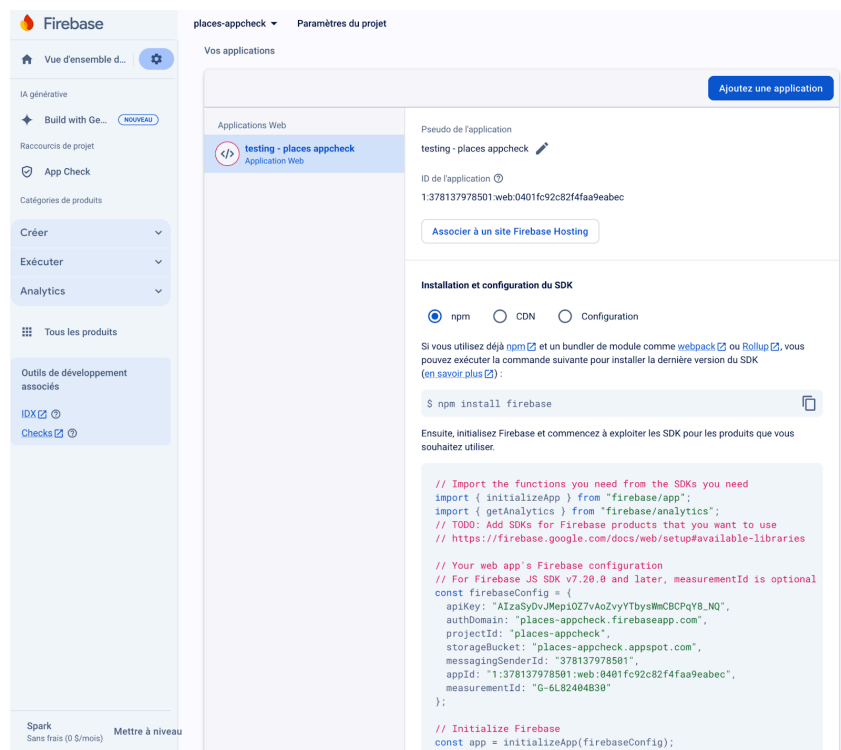
您也可以建立新專案，但請務必在所有 ADC 和 Firebase/reCaptcha 權杖設定中使用該專案。

- 從左上方的「選單」(齒輪) 新增應用程式



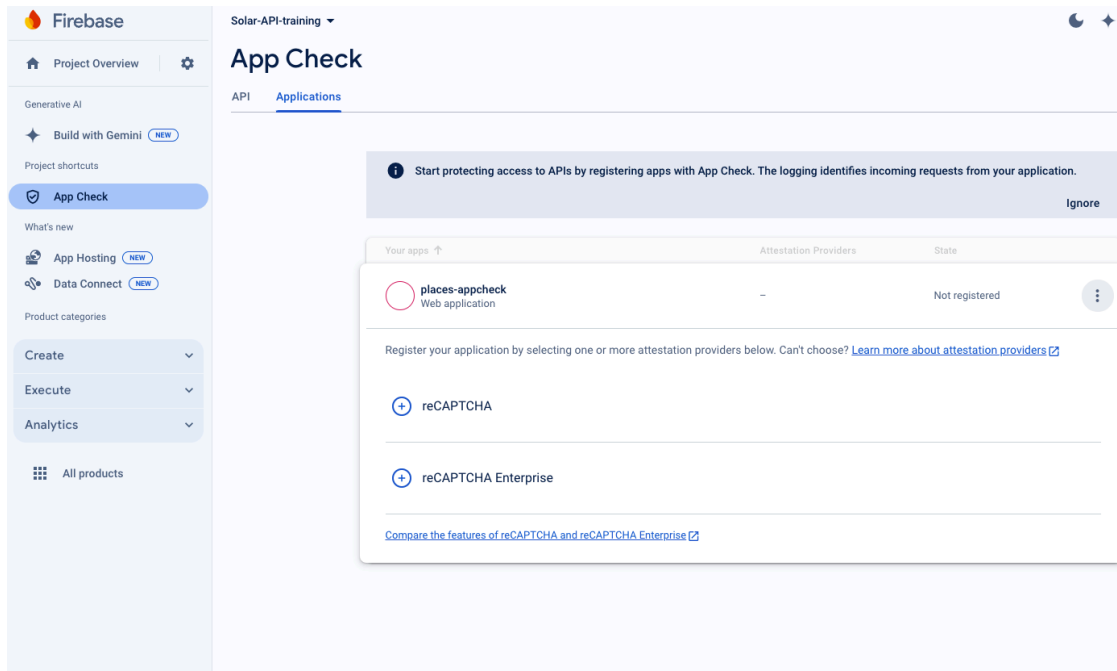
Firestore 初始化程式碼

- 儲存 Firestore 初始化程式碼，以便貼到用戶端的 script.js (下一章)



- 註冊應用程式，允許 Firestore 使用 reCAPTCHA v3 權杖

https://console.firebase.google.com/u/0/project/YOUR_PROJECT/appcheck/apps



- 選擇 reCAPTCHA → 在 reCAPTCHA 網站中建立金鑰 (並設定正確的網域：應用程式開發的本機主機)

在所有設定中使用相同的 Google Cloud 專案 ID

- 將 reCAPTCHA 密鑰貼到 Firebase App Check 中

 places search

Web App

reCAPTCHA

 Registered



Register your app by choosing one or more attestation providers below. Can't decide? [Learn more about attestation providers](#)

 reCAPTCHA

reCAPTCHA secret key 

Token time to live 

30

minutes

After registering, install the App Check SDK so APIs can verify requests from your apps. [View docs](#)

By registering your web app for App Check your app will use reCAPTCHA v3. Your use of reCAPTCHA v3 is subject to the applicable reCAPTCHA Terms of Service.

Cancel

Save

- 應用程式狀態應會變成綠色

Your apps 

Attestation Providers

State

 places-appcheck

Web application

reCAPTCHA

 Registered



Register your application by selecting one or more attestation providers below. Can't choose? [Learn more about attestation providers](#)

 reCAPTCHA

 reCAPTCHA Enterprise

[Compare the features of reCAPTCHA and reCAPTCHA Enterprise](#)

確認 Firebase 註冊中的密鑰與 reCAPTCHA 中的密鑰相符

5. 試用版應用程式

- 用戶端網頁應用程式：HTML、JavaScript、CSS 檔案
- 伺服器：Node.js 檔案
- 環境 (.env)：API 金鑰
- 設定 (app.yaml)：Google App Engine 部署設定

Node.js 設定：

- 導覽：開啟終端機，然後前往複製專案的根目錄。
- 安裝 Node.js (如有需要)：18 以上版本。

```
node -v # Check installed version
```

- 初始化專案：執行下列指令，初始化新的 Node.js 專案，並保留所有預設設定：

```
npm init
```

- 安裝依附元件：使用下列指令安裝必要的專案依附元件：

```
npm install @googlemaps/places firebase-admin express axios dotenv
```

設定：Google Cloud 專案的環境變數

- 建立環境檔案：在專案的根目錄中，建立名為 `.env` 的檔案。這個檔案會儲存機密設定資料，不應提交至版本管控。
- 填入環境變數：開啟 `.env` 檔案，並加入下列變數，將預留位置換成 Google Cloud 專案的實際值：

```
# Google Cloud Project ID
GOOGLE_CLOUD_PROJECT="your-cloud-project-id"

# reCAPTCHA Keys (obtained in previous steps)
RECAPTCHA_SITE_KEY="your-recaptcha-site-key"
RECAPTCHA_SECRET_KEY="your-recaptcha-secret-key"

# Maps Platform API Keys (obtained in previous steps)
PLACES_API_KEY="your-places-api-key"
MAPS_API_KEY="your-maps-api-key"
```

請確認每行只有一個變數，且鍵在執行階段正確載入

6. 程式碼總覽

index.html

- 載入 Firebase 程式庫，在應用程式中建立權杖

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Places API with AppCheck</title>
  <style></style> </head>
<body>
  <div id="map"></div>

  <!-- Firebase services -->
  <script src="https://www.gstatic.com/firebasejs/9.15.0/firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/9.15.0/firebase-app-check-compat.js">
</script>

  <script type="module" src="./script.js"></script>
  <link rel="stylesheet" href="./style.css">
</body>
</html>
```

script.js

- 擷取 **API 金鑰**：從後端伺服器擷取 Google 地圖和 Firebase App Check 的 API 金鑰。
- 初始化 **Firebase**：設定 Firebase 進行驗證和安全防護。(取代設定 → 請參閱第 4 章)。

Firebase App Check 權杖的有效時間 (30 分鐘至 7 天) 是在 Firebase 控制台中設定，無法透過強制重新整理權杖來變更。

必須使用 Firebase 設定詳細資料進行編輯

- 啟用 **App Check**：啟用 Firebase App Check，驗證傳入要求的真實性。
- 載入 **Google Maps API**：動態載入 Google 地圖 JavaScript 程式庫，以顯示地圖。
- 初始化地圖：建立以預設位置為中心的 Google 地圖。
- 處理地圖點擊：監聽地圖上的點擊，並據此更新聚焦點。
- 查詢 **Places API**：使用 Firebase App Check 授權，向後端 API (`/api/data`) 傳送要求，擷取所點選位置附近的地點 (餐廳、公園、酒吧) 資訊。
- 顯示標記：在地圖上以標記形式繪製擷取的資料，並顯示名稱和圖示。

```

let mapsApiKey, recaptchaKey; // API keys
let currentAppCheckToken = null; // AppCheck token

async function init() {
  try {
    await fetchConfig(); // Load API keys from .env variable

    //////////// REPLACE with your Firebase configuration details
    const firebaseConfig = {
      apiKey: "AIza.....",
      authDomain: "places.....",
      projectId: "places.....",
      storageBucket: "places.....",
      messagingSenderId: "17.....",
      appId: "1:175.....",
      measurementId: "G-CPQ.....",
    };
    //////////// REPLACE

    // Initialize Firebase and App Check
    await firebase.initializeApp(firebaseConfig);
    await firebase.appCheck().activate(recaptchaKey);

    // Get the initial App Check token
    currentAppCheckToken = await firebase.appCheck().getToken();

    // Load the Maps JavaScript API dynamically
    const scriptMaps = document.createElement("script");
    scriptMaps.src = `https://maps.googleapis.com/maps/api/js?
key=${mapsApiKey}&libraries=marker,places&v=beta`;
    scriptMaps.async = true;
    scriptMaps.defer = true;
    scriptMaps.onload = initMap; // Create the map after the script loads
    document.head.appendChild(scriptMaps);
  } catch (error) {
    console.error("Firebase initialization error:", error);
    // Handle the error appropriately (e.g., display an error message)
  }
}

window.onload = init()

// Fetch configuration data from the backend API
async function fetchConfig() {
  const url = "/api/config";

  try {
    const response = await fetch(url);
    const config = await response.json();
    mapsApiKey = config.mapsApiKey;
    recaptchaKey = config.recaptchaKey;
  } catch (error) {
    console.error("Error fetching configuration:", error);
  }
}

```

```

    // Handle the error (e.g., show a user-friendly message)
  }
}

// Initialize the map when the Maps API script loads
let map; // Dynamic Map
let center = { lat: 48.85557501, lng: 2.34565006 };
function initMap() {
  map = new google.maps.Map(document.getElementById("map"), {
    center: center,
    zoom: 13,
    mapId: "b93f5cef6674clff",
    zoomControlOptions: {
      position: google.maps.ControlPosition.RIGHT_TOP,
    },
    streetViewControl: false,
    mapTypeControl: false,
    clickableIcons: false,
    fullscreenControlOptions: {
      position: google.maps.ControlPosition.LEFT_TOP,
    },
  });

  // Initialize the info window for markers
  infoWindow = new google.maps.InfoWindow({});

  // Add a click listener to the map
  map.addListener("click", async (event) => {
    try {
      // Get a fresh App Check token on each click
      const appCheckToken = await firebase.appCheck().getToken();
      currentAppCheckToken = appCheckToken;

      // Update the center for the Places API query
      center.lat = event.latLng.lat();
      center.lng = event.latLng.lng();

      // Query for places with the new token and center
      queryPlaces();
    } catch (error) {
      console.error("Error getting App Check token:", error);
    }
  });
}

function queryPlaces() {
  const url = '/api/data'; // "http://localhost:3000/api/data"

  const body = {
    request: {
      includedTypes: ['restaurant', 'park', 'bar'],
      excludedTypes: [],
    },
  };
}

```

```

    maxResultCount: 20,
    locationRestriction: {
      circle: {
        center: {
          latitude: center.lat,
          longitude: center.lng,
        },
        radius: 4000,
      },
    },
  },
};

// Provides token to the backend using header: X-Firebase-AppCheck

fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Firebase-AppCheck': currentAppCheckToken.token,
  },
  body: JSON.stringify(body),
})
  .then((response) => response.json())
  .then((data) => {
    // display if response successful
    displayMarkers(data.places);
  })
  .catch((error) => {
    alert('No places');
    // eslint-disable-next-line no-console
    console.error('Error:', error);
  });
}

///// display places markers on map
...
```

server.js

- 從 `.env` 檔案載入環境變數 (API 金鑰、Google 專案 ID)。
- 啟動伺服器，監聽 `http://localhost:3000` 上的要求。
- 使用應用程式預設憑證 (ADC) 初始化 **Firebase Admin SDK**。
- 從 `script.js` 接收 reCAPTCHA 權杖。
- 驗證收到的權杖是否有效。
- 如果權杖有效，會向 **Google Places API** 發出 **POST** 要求，其中包含搜尋參數。
- 處理 **Places API** 的回應並傳回給用戶端。

```

const express = require('express');
const axios = require('axios');

const admin = require('firebase-admin');

// .env variables
require('dotenv').config();

// Store sensitive API keys in environment variables
const recaptchaSite = process.env.RECAPTCHA_SITE_KEY;
const recaptchaSecret = process.env.RECAPTCHA_SECRET_KEY;
const placesApiKey = process.env.PLACES_API_KEY;
const mapsApiKey = process.env.MAPS_API_KEY;

// Verify environment variables loaded (only during development)
console.log('recaptchaSite:', recaptchaSite, '\n');
console.log('recaptchaSecret:', recaptchaSecret, '\n');

const app = express();
app.use(express.json());

// Firebase Admin SDK setup with Application Default Credentials (ADC)
const { GoogleAuth } = require('google-auth-library');
admin.initializeApp({
  // credential: admin.credential.applicationDefault(), // optional: explicit ADC
});

// Main API Endpoint
app.post('/api/data', async (req, res) => {
  const appCheckToken = req.headers['x-firebase-appcheck'];

  console.log("\n", "Token", "\n", "\n", appCheckToken, "\n")

  try {
    // Verify Firebase App Check token for security
    const appCheckResult = await admin.appCheck().verifyToken(appCheckToken);

    if (appCheckResult.appId) {
      console.log('App Check verification successful!');
      placesQuery(req, res);
    } else {
      console.error('App Check verification failed.');
```

```

      res.status(403).json({ error: 'App Check verification failed.' });
    }
  } catch (error) {
    console.error('Error verifying App Check token:', error);
    res.status(500).json({ error: 'Error verifying App Check token.' });
  }
});

// Function to query Google Places API
async function placesQuery(req, res) {
```



```

console.log('#####');
console.log('\n', 'placesApiKey:', placesApiKey, '\n');

const queryObject = req.body.request;
console.log('\n', 'Request', '\n', '\n', queryObject, '\n')

const headers = {
  'Content-Type': 'application/json',
  'X-Goog-FieldMask': '*',
  'X-Goog-API-Key': placesApiKey,
  'Referer': 'http://localhost:3000', // Update for production(ie.: req.hostname)
};

const myUrl = 'https://places.googleapis.com/v1/places:searchNearby';

try {
  // Authenticate with ADC
  const auth = new GoogleAuth();
  const { credential } = await auth.getApplicationDefault();

  const response = await axios.post(myUrl, queryObject, { headers, auth: credential });

  console.log('##### SUCCESS', '\n', '\n', 'Response', '\n', '\n', );
  const myBody = response.data;
  myBody.places.forEach(place => {
    console.log(place.displayName);
  });
  res.json(myBody); // Use res.json for JSON data
} catch (error) {
  console.log('##### ERROR');
  // console.error(error); // Log the detailed error for debugging
  res.status(error.response.status).json(error.response.data); // Use res.json for errors
too
}
}

// Configuration endpoint (send safe config data to the client)
app.get('/api/config', (req, res) => {
  res.json({
    mapsApiKey: process.env.MAPS_API_KEY,
    recaptchaKey: process.env.RECAPTCHA_SITE_KEY,
  });
});

// Serve static files
app.use(express.static('static'));

// Start the server
const port = process.env.PORT || 3000;
app.listen(port, () => {
  console.log(`Server listening on port ${port}`, '\n');
});

```

```
});
```

步驟 7 · 約 5 分鐘

7. 執行應用程式

在所選環境中，從終端機執行伺服器，然後前往 <http://localhost:3000>

```
npm start
```

系統會將權杖建立為全域變數，隱藏在使用者瀏覽器視窗中，並傳輸至伺服器進行處理。如要查看權杖詳細資料，請參閱伺服器記錄。

Token
eyJraWQioiIjNjbjVDS1EiLCJ0eXA0IjKV1QlCLjhbGciOiJSUzI1NiJ9.eyJz
dWIiOi1xOTQlExODIyNzQxMjQ1QjQ3ZWZlNjBGNjQ3ZDYwMmM2MzN4NDU0Ny
IsImF1ZCI6WyJwcm9qZWNoIiwuMTE4MjI3NDYyNDg2IiwicHJvamVjdHnCL3Nv
bGFnLWwFwaS10cmFpbmluZyZlJCJwcm92aWRkcCI6IjNjLlY2FwdGNoYVY9M2MyIsIm
lucyLW16Imh0dHBzO1wvXzYyZjYlYmFzZWZwGNoZmNlMmdvb2ZzZWZwaXMuY29t
XC8xMTgyMjc0MTE0ODYiLCJleHAiOiJMTk4NzEzMDIsImh0dCI6MTcxOTg2OT
UwMiwiianRpIjojQWY4MHZkXzhmU1pibWQ3VFZ0NS1WnkZpNFNlSjZSjZjAtKw4
dUxaVW1oMCJ9.WrvWcLV4VpHe3mL0n06pog4x8wgL8g6c_0gBFXABkcdv1ZPF
6I8G5ZDMwMrB3ayitrhiQK4XND4qyu8dLILXqKzaUmemb8o8IDLISUcxkjaMqbX
DYBsCW4fAqgdx37H5wFRA3uatZr9j-7U91bf9GUpmKvN5PjwD125kKZ0DR8X
5lqxdf7apcxdxjYI0t5H-qS-HbFAyYhm8pEOyx-Rs-6VLX512g26_uXjijj8Y-
Fu_wa4YiWobFdYl8bl1t5eVtVeB9Q2xiYv0FZEPgufvqfBg9pmmSLofZT5W
J970_Z0G6vTrutN1dhqxZ278dGiugb1h9NpccY4LstfPRCTcVljivFWFR8yzS
vYA_nltRS11BWA6a1ymWFFLPpyBjpt3y3NzKws-XI-tI2ws0tN1Ji0xiXUA6CtjS
WTwmLxfCfpceecRB_xCP7pJ3Q2PT0BZ2KP0Gkp8aFmC9A1I1Slsb2Tntes
MpmdtIavoi_Eoe6dhx_TxaPoy0j9R
App Check token verified
#####

如要瞭解伺服器的函式和對 Places API Nearby Search 要求的相關回應，請參閱伺服器記錄。

```
queryObject:
{
  includedTypes: [ 'restaurant', 'park', 'bar' ],
  excludedTypes: [],
  maxResultCount: 20,
  locationRestriction: { circle: { center: [Object], radius: 4
000 } }
}

##### SUCCESS
places names:

{ text: 'Jardin du Luxembourg', languageCode: 'en' }
{ text: 'Tuileries Garden', languageCode: 'en' }
{ text: 'Place des Vosges', languageCode: 'en' }
{ text: 'Jardin des Plantes', languageCode: 'en' }
{ text: 'Café de Flore', languageCode: 'en' }
{ text: 'Westfield Forum des Halles', languageCode: 'en' }
{ text: 'Ikea City', languageCode: 'en' }
{ text: 'Bateaux-Mouches', languageCode: 'en' }
{ text: 'Champ de Mars', languageCode: 'en' }
{ text: 'Food Market / Chill Zone / La Felicità', languageCode
: 'en' }
{ text: 'Angelina', languageCode: 'en' }
{ text: 'Bouillon Chartier Grands Boulevards', languageCode: '
en' }
```

疑難排解：

請確保設定中的 Google 專案 ID 一致：

- 在 .env 檔案中 (GOOGLE_CLOUD_PROJECT 變數)
- 在終端機 gcloud 設定中：

```
gcloud config set project your-project-id
```

- 在 reCaptcha 設定中

Google reCAPTCHA

Add advanced features such as the **MEFA**, protection against spam and fraud, and in Google Cloud.

- ✓ Up to 10,000 evaluations per month, free of charge
- ✓ No credit card required

Text ⓘ

test codelab - places-appcheck

30 / 50

Type of reCAPTCHA ⓘ

☒ Based on score (v3) Validate queries with a score

☐ Challenge (v2) Validate queries using a test

Domains ⓘ

... Add a domain, e.g. example.com

Google Cloud Platform

It looks like you've already used Google Cloud. To start with, we're going to create a project and activate the necessary APIs.

Name of project

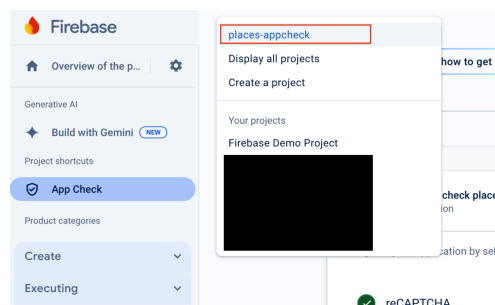
places-appcheck

15 / 30

CLOUD PLATFORM LEATHER

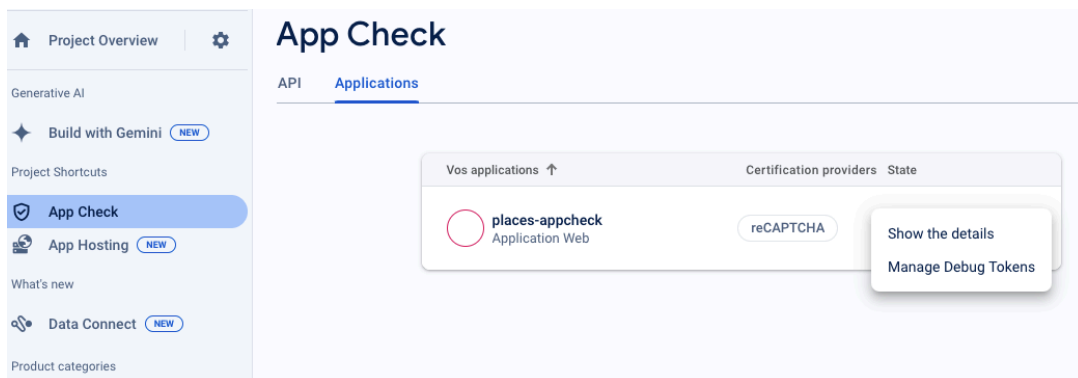
CANCEL SEND

- 在 Firebase 設定中



其他

- 建立偵錯權杖，在 `script.js` 中取代 reCAPTCHA 網站金鑰，用於測試和疑難排解。



```

try {
  // Initialize Firebase first
  await firebase.initializeApp(firebaseConfig);
  // Set the debug token
  if (window.location.hostname === 'localhost') { // Only in development
    await firebase.appCheck().activate(
      'YOUR_DEBUG_FIREBASE_TOKEN', // Replace with the token from the console
      true // Set to true to indicate it's a debug token
    );
  } else {
    // Activate App Check
    await firebase.appCheck().activate(recaptchaKey);
  }
}

```

- 如果驗證失敗次數過多 (例如使用錯誤的 reCAPTCHA 網站金鑰)，可能會觸發暫時節流。

```

FirebaseError: AppCheck: Requests throttled due to 403 error. Attempts allowed again after
01d:00m:00s (appCheck/throttled).

```

ADC 憑證

- 確認使用正確的 gcloud 帳戶

```
gcloud auth login
```

- 確認已安裝必要的程式庫

```
npm install @googlemaps/places firebase-admin
```

- 確認 [server.js](#) 中已載入 Firebase 程式庫

```
const {GoogleAuth} = require('google-auth-library');
```

- [本機開發](#)：設定 ADC

```
gcloud auth application-default login
```

- 模擬：已儲存 ADC 憑證

```
gcloud auth application-default login --impersonate-service-account
your_project@appspot.gserviceaccount.com
```

- 最後在本機測試 ADC，將下列指令碼儲存為 test.js，並在終端機中執行：`node test.js`

```
const {GoogleAuth} = require('google-auth-library');

async function requestTestADC() {
  try {
    // Authenticate using Application Default Credentials (ADC)
    const auth = new GoogleAuth();
    const {credential} = await auth.getApplicationDefault();

    // Check if the credential is successfully obtained
    if (credential) {
      console.log('Application Default Credentials (ADC) loaded successfully!');
      console.log('Credential:', credential); // Log the credential object
    } else {
      console.error('Error: Could not load Application Default Credentials (ADC).');
    }

    // ... rest of your code ...

  } catch (error) {
    console.error('Error:', error);
  }
}

requestTestADC();
```

8. 這樣就完成了，做得好！

後續步驟

部署至 App Engine：

- 準備將專案部署至 Google App Engine，並視需要變更設定。
- 使用 `gcloud` 指令列工具或 App Engine 控制台部署應用程式。

強化 Firebase 驗證：

- **預設權杖與自訂權杖**：導入 Firebase 自訂權杖，進一步使用 Firebase 服務。
- **權杖生命週期**：設定適當的權杖生命週期，敏感作業的權杖生命週期應較短 (自訂 Firebase 權杖最長一小時)，一般工作階段的權杖生命週期應較長 (reCAPTCHA 權杖：30 分鐘到 7 小時)。
- **探索 reCAPTCHA 的替代方案**：評估 DeviceCheck (iOS)、SafetyNet (Android) 或 App Attest 是否符合您的安全需求。

整合 Firebase 產品：

- **即時資料庫或 Firestore**：如果應用程式需要即時資料同步處理或離線功能，請與即時資料庫或 Firestore 整合。
- **Cloud Storage**：使用 Cloud Storage 儲存及提供使用者原創內容，例如圖片或影片。
- **驗證**：運用 Firebase 驗證建立使用者帳戶、管理登入工作階段，以及處理密碼重設作業。

擴展至行動裝置：

- **Android 和 iOS**：如果您打算推出行動應用程式，請為 Android 和 iOS 平台分別建立版本。
 - **Firebase SDK**：使用 Android 和 iOS 版 Firebase SDK，將 Firebase 功能無縫整合至行動應用程式。
-